Applied Intelligence (2025) 55:1145
https://doi.org/10.1007/s10489-025-07019-7

CARM-Diff: a cross-attention guided diffusion model for game trajectory prediction

Zhiheng Zhang[1] · Lina Lu[1] · Chushu Yi[1] · Tingting Wei[1] · Jing Chen[1]

Received: 6 August 2025 / Accepted: 16 November 2025 / Published online: 15 December 2025
© The Author(s) 2025

Abstract
In real decision-making scenarios involving multi-agent games, such as intelligent unmanned systems, military confrontations, and autonomous navigation, the coupling of participant strategies and the incompleteness of perceived information make the accurate inference of dynamic game trajectories a critical and challenging task. To address this problem, this paper proposes a game trajectory modeling method that integrates a cross-attention mechanism with a moving diffusion process, termed CARM-Diff (Cross-Attention Autoregressive Moving Diffusion Model). The model combines autoregressive structures with cross-attention to capture the temporal evolution of sequences while explicitly modeling strategic interactions between agents. We design a lightweight feature extraction module and, leveraging the Markov property of diffusion models, introduce a deterministic evolution process of historical states to simulate noise, thereby enhancing the model's capability to learn local temporal patterns. Meanwhile, a cross-attention mechanism is introduced in the reverse diffusion stage to guide the model in focusing on the opponent's historical sequential behavior, enabling more precise capture of inter-agent influences. Furthermore, we design a residual gated trajectory modeling structure that fuses the agent's own behavioral evolution with interaction effects induced by opponents. Gating factors are dynamically generated through multilayer perceptrons to achieve adaptive information fusion. We construct a dynamic trajectory dataset based on an underwater pursuit-evasion game to validate our approach, and the proposed CARM Diff framework is generalizable to a wide range of multi-agent interactive systems. Experimental results show that CARM-Diff outperforms mainstream baseline methods in both prediction accuracy and dynamic interaction modeling, demonstrating the effectiveness and practical potential of the proposed model.

Keywords Game trajectory · Cross-attention mechanism · Diffusion model · Residual gated structure

1 Introduction

With the increasing strategic significance of oceans, the application of underwater vehicles in military defense, marine resource development, and ecological protection is becoming increasingly important [1]. In complex adversarial environments, particularly in underwater anti-submarine interception, ecological reserve patrolling, and underwater facility defense, the performance and decision-making capability of underwater vehicles are crucial [2]. The efficient operation of underwater vehicles requires accurate trajectory prediction to achieve optimal path planning and autonomous decision-making. However, due to the high coupling of strategies and the incompleteness of information among participants in a game-theoretic adversarial environment, inferring the dynamic trajectories during the game process becomes a highly challenging task.

✉ Lina Lu
lulina16@nudt.edu.cn

Zhiheng Zhang
zhangzhiheng19@nudt.edu.cn

Chushu Yi
yichushu61@nudt.edu.cn

Tingting Wei
weitingting20@nudt.edu.cn

Jing Chen
chenjing001@vip.sina.com

[1] College of Intelligence Science and Technology, National University of Defense Technology, No. 1 Fuyuan Rd, Changsha 410073, Hunan, China

In recent years, with the rapid development of advanced data acquisition systems, high-performance computing platforms, and artificial intelligence technologies, the precision of trajectory prediction for underwater vehicles has significantly improved. In particular, deep learning methods have made substantial progress in time series data modeling, greatly enhancing the autonomous navigation and mission execution capabilities of underwater vehicles. However, in adversarial game environments, vehicles not only need to make dynamic decisions based on their own objectives but must also respond in real-time to opponents' behavior patterns [3], forming complex and highly coupled strategic interaction systems. Traditional trajectory prediction methods [4–6] often focus on modeling the behavior of a single agent, making it challenging to address the complex dynamic interactions and strategy evolution in multi-agent games. Therefore, accurately inferring the trajectories during the game process has become a core challenge and is key to enhancing the intelligent decision-making capabilities of underwater vehicles.

Accurate inference of game trajectories not only aids underwater vehicles in optimizing their own behavior but also effectively predicts the future actions of opponents, enhancing the adaptability and anti-jamming capabilities of autonomous systems. In this context, trajectory prediction methods in game environments have broad applications in areas such as task planning and decision support [7, 8]. To address the shortcomings of existing methods that fail to effectively capture multi-agent dynamic interactions and strategy dependencies in complex game environments, this paper proposes a game trajectory modeling method based on a cross-attention mechanism and a moving diffusion process—CARM-Diff (Cross-Attention Autoregressive Moving Diffusion Model). This model integrates autoregressive modeling and cross-attention mechanisms, enabling it to capture the strategic interactions among game participants while modeling the temporal behavior of the agents and effectively inferring future trajectories in dynamic games. We establish an underwater vehicle evasion model based on iterative optimal responses to generate a dynamic game trajectory dataset to validate the effectiveness of the CARM-Diff model. The core innovations of this paper are reflected in the following three aspects:

1. We have constructed a lightweight feature extraction module that effectively captures local temporal patterns. Based on the Markov chain characteristics of the diffusion process, we simulate the noise process using deterministic combinations of historical states, integrating latent space optimization with temporal modeling to mitigate the impact of random disturbances.
2. We introduced a cross-attention mechanism in the diffusion process to model the temporal interaction relationships between game participants and their opponents. By computing attention weights to guide the reverse denoising process, we enhance the model's ability to express the dynamic behaviors of multiple agents.
3. We proposed a residual gated trajectory modeling structure that integrates the linear evolutionary trends of game participants with the characteristic information of their opponents. This structure dynamically generates gating factors using a multi-layer perceptron, enabling adaptive information fusion and significantly improving the accuracy of sequence predictions.

Compared with existing diffusion-based time series models, CARM-Diff extends the diffusion paradigm from single-sequence modeling to multi-agent interactive trajectory inference, introducing attention-guided conditional diffusion and adaptive residual gating to better capture inter-agent influences. Beyond underwater games, the proposed CARM-Diff model offers a general framework for multi-agent interaction modeling and trajectory forecasting. Its cross-attention-guided diffusion process can be directly applied to diverse domains such as autonomous driving, aerial combat simulation, cooperative robotics, human–robot interaction, and financial market strategy prediction, where entities engage in strategic behaviors with partial information. The remainder of this paper is organized as follows: Section 2 provides a brief overview of related works. Section 3 describes the methods employed in this study. Section 4 presents the experimental process and discusses the experimental results. Section 5 concludes the paper.

2 Related works

This section provides an overview of trajectory prediction research in the game process, as well as the relevant applications of diffusion models and attention mechanisms.

2.1 Trajectory prediction in the game process

In complex game environments, the behavior of game agents depends on feedback and inferences from historical actions, while also being influenced by other game entities. Currently, trajectory prediction methods in the game process can be broadly categorized into the following types. One type is based on traditional game theory for strategy modeling, predicting the behavior trajectories of participants through equilibrium solutions of the game [8–10]. These methods often rely on static solutions like Nash equilibrium to infer participants' strategic behaviors and further predict their future trajectories. Another type of method combines reinforcement learning and imitation learning frameworks

[11–13], dynamically adjusting participants' decision-making strategies by learning from historical behavior data during the game process. These methods better model the complex interactions and long-term strategy evolution among participants in multi-agent game environments. Furthermore, there are studies that utilize deep learning and neural network technologies to extract patterns directly from trajectory data in the game process through end-to-end learning methods [14, 15], thus achieving precise trajectory predictions in a higher-dimensional strategy space.

At the algorithmic level, data generated during the game process is typically processed using time series prediction methods, employing deep neural networks for inference. These methods effectively handle temporal dependencies and capture the behavioral evolution patterns of game agents. With the introduction of generative model technologies, trajectory prediction methods in the game process have achieved significant improvements in both accuracy and generalization capability. In recent years, generative models based on Generative Adversarial Networks (GAN) and diffusion processes have demonstrated powerful predictive capabilities in multi-agent game environments, allowing for more accurate simulation and generation of game trajectories.

2.2 Diffusion models in dynamic systems

Denoising Diffusion Probabilistic Models (DDPM) are a type of generative model based on diffusion processes [16]. The core idea is to learn the data distribution through a two-stage process of forward noise addition and reverse denoising. In the forward phase, the model progressively adds Gaussian noise to the data using a Markov chain, causing it to eventually degrade into an approximate Gaussian distribution. In the reverse phase, the model gradually denoises and reconstructs the data by training a noise prediction network, optimizing the network by minimizing the discrepancy between the predicted noise and the actual noise.

Diffusion models represent a significant breakthrough in the field of generative models and are gradually expanding their application scope to complex temporal tasks such as dynamic system modeling and prediction. Initially utilized for high-quality image generation, diffusion models demonstrate superior modeling capabilities compared to traditional GAN and VAE by progressively introducing noise and learning the reverse denoising process to approximate the data distribution. In recent years, researchers have begun to extend these models to dynamic systems, particularly showcasing great potential in tasks such as nonlinear system modeling, trajectory prediction, and multi-agent interaction modeling. A typical method, such as DiffusionTS [17], defines the diffusion process in the time series data space, enabling the capture of multimodal future trajectory distributions and overcoming the limitations of traditional models in uncertainty modeling. Additionally, some studies [18, 19] combine diffusion models with physical priors to learn the evolutionary laws of physical dynamical systems, enhancing the interpretability and stability of the models regarding system structures. Compared to classical dynamic system modeling methods (such as Kalman filtering, state-space models, RNNs, etc.), diffusion models possess stronger capabilities in modeling the evolution of time series [4].

2.3 Attention-based interactive modeling

The attention mechanism [20] is one of the methods in deep learning for processing complex sequence data and modeling multidimensional interactions. The core idea is to automatically focus on the most important parts of the input information through a dynamic weight allocation mechanism, effectively capturing key information. This mechanism is inspired by the human visual attention system and aims to enhance the model's perception and decision-making abilities in complex tasks by dynamically adjusting the focus on different parts of the input. In this way, the attention mechanism enables the model to selectively attend to relevant features amid vast amounts of information while ignoring unimportant or redundant data, thereby significantly improving the efficiency and accuracy of task processing.

The advantages of the attention mechanism are particularly pronounced in multi-agent systems and dynamic environments. In tasks such as social navigation [21], traffic trajectory prediction [5, 22], and group behavior modeling [23, 24], models need not only to understand individual behavior patterns but also to consider complex, asymmetric, multiscale, and time-varying interactions among individuals. Especially with self-attention and multi-head attention, models can adaptively capture interactions between different agents, thereby enhancing sensitivity to environmental changes and predictive capabilities. The Transformer architecture [6, 25, 26], as a representative model based on the attention mechanism, has become an important tool in time series modeling and complex system modeling, continually refined in various studies.

In summary, existing game trajectory prediction methods based on game theory or learning paradigms focus primarily on static equilibria or long-term policy optimization, often lacking fine-grained temporal interaction modeling. Diffusion-based generative models have demonstrated strong capability in time-series representation, yet they typically inject random noise without explicitly capturing inter-agent dependencies. Meanwhile, attention-based frameworks effectively model relational dynamics but rarely incorporate temporal stochasticity and sequential evolution in a unified manner. Distinct from these approaches, the proposed

CARM-Diff model integrates a deterministic moving diffusion process with a cross-attention mechanism to simultaneously capture temporal evolution trends and interactive strategic dependencies among agents. This hybrid design enables CARM-Diff to achieve more stable trajectory inference in complex game environments.

3 Methodology

This section describes the methods supporting this paper, including how we constructed the game trajectory dataset, the theoretical derivation for predictions using CARM-Diff, and the designed training objectives.

3.1 Game trajectory dataset construction

Consider a discrete-time trajectory game with N participants over a time span of T. At each discrete time step $t \in [0, T]$, each participant i aims to minimize a cost function J_i by controlling the input $u_i^t \in \mathbb{R}^{m_i}$ to alter their own state $x_i^t \in \mathbb{R}^{n_i}$. The state evolution of each participant follows independent dynamical systems denoted as $x_i^{t+1} = f_i\left(x_i^t, u_i^t\right)$.

Define the aggregated representation of participant number dimensions and time dimensions as $x^t := (x_1^t, ..., x_N^t)$, $U_i := \left(u_i^1, ..., u_i^T\right)$, and $X := \left(x^1, ..., x^T\right)$. Starting from a given initial state $\hat{x}^0 := \left(\hat{x}_1^0, ..., \hat{x}_N^0\right)$, each participant i aims to solve the following optimization problem:

$$\forall i \in [N] \begin{cases} \min\limits_{X_i, U_i} J_i\left(X, U_i; \theta_i\right) \\ \quad \text{s.t.} x_i^{t+1} = f_i\left(x_i^t, u_i^t\right) \\ \qquad x_i^0 = \hat{x}_i^0 \geqslant 0 \\ \qquad {}^p g\left(X_i, U_i\right) \geq 0 \\ \qquad {}^s g(X, U) \geq 0. \end{cases} \tag{1}$$

where $\forall t \in [T-1]$, ${}^p g$ and ${}^s g$ are the private constraints and shared constraints that all participants need to follow, and $J_i(\cdot)$ is the cost function of participant i, parameterized by a vector $\theta^i \in \mathbb{R}^{k_i}$ composed of multiple basis functions. The parameters can be either linear or nonlinear, and neural networks can be used for nonlinear parameter design; however, this approach may reduce interpretability and should be carefully designed based on the specific application context. That is, $J_i(\cdot; \theta_i) \equiv \sum_{t=1}^{T} g_i^t(\cdot; \theta_i)$, $g_i^t = \sum_{j=1}^{l} w_i^j g_i^{j,t}$, and w_i^j are the weights of the j-th basis function in the cost function of participant i (with $\sum_{j=1}^{l} w_i^j = 1$).

The solution $U^* := \left(U_1^*, ..., U_N^*\right)$ of this generalized Nash equilibrium problem satisfies $J_i\left(X^*, U_i^*; \theta_i\right) \leqslant J_i\left(\left(X_i, X_{\neg i}^*\right), U_i; \theta_i\right)$ and holds for all participants under feasible deviations (X_i, U_i), where $X_{\neg i}$ represents the states excluding participant i. Generally, the existence of a global Nash equilibrium presents challenges in solving, so we only require the condition of local optimality to hold. The core idea is that, at each time step t, each agent i optimizes its control strategy U_i^t while assuming the strategies of all other agents U_{-i}^t are fixed from the previous iteration. This process iterates until the strategies of all agents converge. The detailed procedure for generating game trajectory is outlined in Algorithm 1.

3.2 Diffusion process modeling methods

This subsection covers the principles of CARM-Diff, including a general description of the model framework, feature extraction for game trajectories, inferring evolutionary trends through the forward diffusion process, and performing reverse diffusion by combining cross-attention mechanisms and gating mechanisms.

3.2.1 Model overall description

We referred to some advanced prediction methods based on diffusion models, such as Diffusion-TS [17] and ARMD [4], and built upon them to propose the CARM-Diff model, which aims to infer future trajectories in the game process. Diffusion-TS introduces interpretable seasonal-trend decomposition within the diffusion framework, focusing on disentangled representation learning for general time series generation. However, it primarily handles single-sequence temporal synthesis without explicitly considering inter-agent dependencies. ARMD replaces random Gaussian noise with a deterministic sliding mechanism inspired by Auto-Regressive Moving Average (ARMA) theory, aligning the diffusion process with the continuous evolution of time series. Yet, it remains an unconditional model and cannot capture dynamic interactions between multiple agents. Unlike previous diffusion-based forecasting models, CARM-Diff is designed for game trajectory prediction, where the evolution of one agent's trajectory depends on both its own dynamics and its opponent's strategies. Our proposed CARM-Diff extends deterministic diffusion to multi-agent interactive prediction by introducing a cross-attention-guided conditional diffusion process and a residual-gated fusion mechanism. This design enables explicit modeling of strategy coupling between agents while maintaining the temporal continuity advantages of ARMD. The training and inference process of the CARM-Diff model is illustrated in Fig. 1. We categorize the two sides of the game into a pursuer and an evader. From the perspective of the evader, we infer the trajectory $\tau^{(c)} = s_t^{(c)}$ of the pursuer in the future time domain $T = \{t_1, t_2, ..., t_P\}$. By

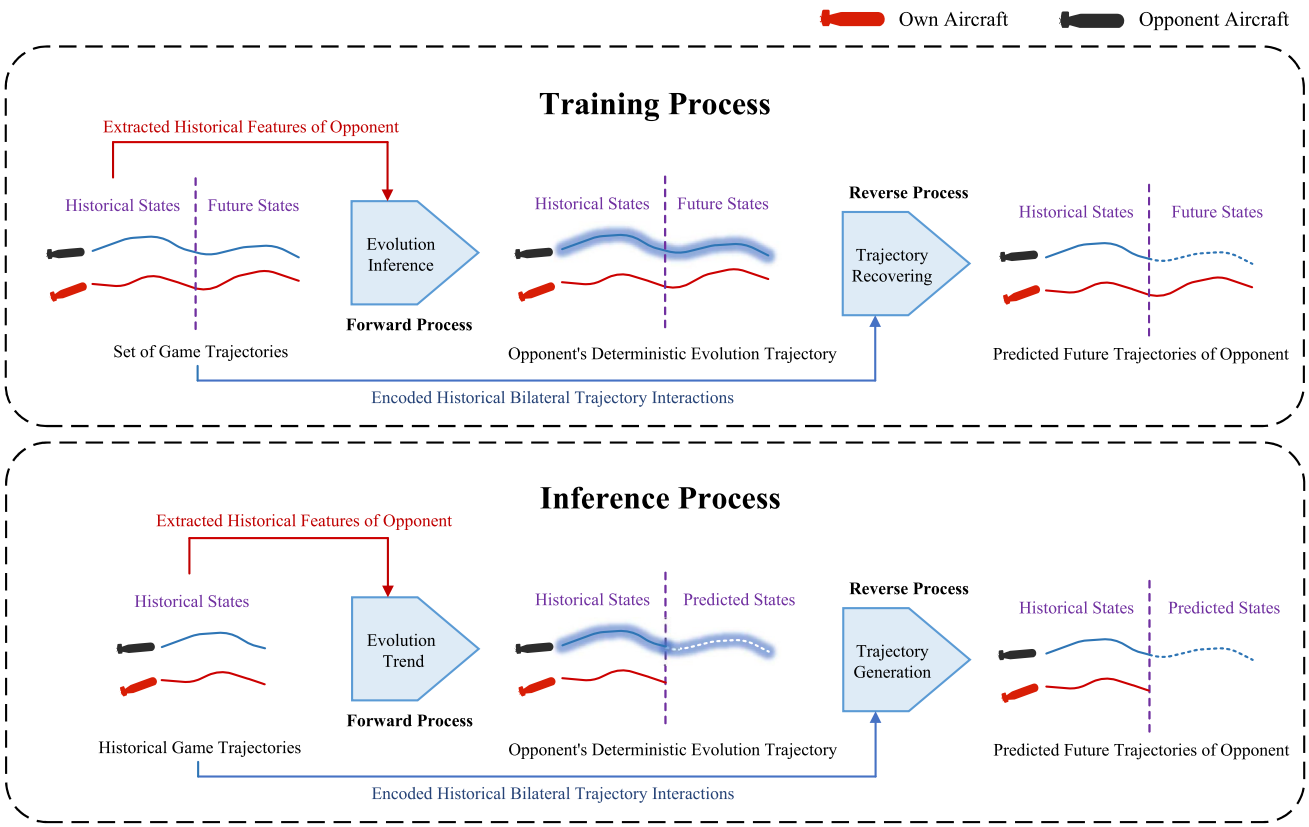

Fig. 1 Training and inference process of CARM-Diff

analyzing the trajectory data $H = \{\tau^{(k)}\}_{k=1}^{K}$ generated by both parties under historical equilibrium strategies, we construct a conditional diffusion process to trajectory prediction. Specifically, in the training phase, we extract the historical features of both sides of the game and replace the random noise injection in the forward process with a deterministic evolutionary trend. In the reverse process, we utilize a cross-attention mechanism and gating modules to fuse the evolutionary trends and interaction features, thereby predicting future trajectories. In the following sections, we will provide a detailed explanation of the feature extraction component of CARM-Diff as well as the forward and reverse processes.

3.2.2 Feature extraction

The extraction and modeling of spatiotemporal features play a crucial role in complex time series prediction tasks. For instance, Wood [27] analyzed varicella-zoster case number trends and showed that incorporating both spatial and temporal attributes significantly enhanced predictive accuracy in epidemiological forecasting tasks. To enhance the model's ability to model the input data, this paper designs a lightweight yet powerful feature extraction module. This module combines linear transformation, normalization, convolutional structures, and residual connections, effectively capturing local temporal patterns while maintaining feature stability.

Let the input tensor be $x \in \mathbb{R}^{B\times T\times C}$, where B represents the batch size, T denotes the time steps, and C indicates the number of channels. First, we map the feature dimensions through a linear layer and apply Layer Normalization to enhance the stability of the feature distribution, resulting in x_1. Rearranging the input from (B, T, C) to (B, C, T) to accommodate the one-dimensional convolution operation, resulting in x_2. Next, two layers of convolution are used to extract local temporal information. The first layer expands the number of channels, resulting in x_3, while the second layer compresses the channels to restore the original dimensions, resulting in x_4. Rearrange the dimensions back to (B, T, C) and add it to the original input to form a residual connection, followed by another normalization process, resulting in $\hat{x}$:

$$\begin{cases} x_1 = \mathrm{LayerNorm}(\mathrm{Linear}(x)) \\ x_2 = \mathrm{Permute}(x_1) \in \mathbb{R}^{B\times C\times T} \\ x_3 = \mathrm{Conv1D}_{k_1}(x_2) \\ x_4 = \mathrm{Conv1D}_{k_2}(\mathrm{GELU}(x_3)) \\ \hat{x} = \mathrm{LayerNorm}(\mathrm{Permute}(x_4) + x), \end{cases} \tag{2}$$

the final output has the same dimensions as the input tensor but possesses enhanced local modeling capability and better numerical stability. We denote the network in (2) as $\mathrm{FE}(\cdot)$. This module serves as a fundamental feature extraction unit within the overall model architecture, providing high-quality local feature representations for the subsequent cross-attention mechanism.

Algorithm 1 Game Trajectory Generation via Iterative Optimal Responses.

Require: Initial states $\hat{x}^0$; Cost functions J_i; Dynamics f_i; Maximum iterations $K_{\max}$; Termination distance d_{term}.
Ensure: Game trajectory $X = (x^1, \ldots, x^T)$.
Initialize the game: set $t = 0$, $X[0] = \hat{x}^0$.
for $t = 0$ to $T - 1$ **do**
  Initialize strategies $U_i^{(0)}$ for all agents i.
  $k \leftarrow 0$
  repeat
    for each agent $i = 1$ to N **do**
      Fix other agents' strategies $U_{-i}^{(k)}$.
      Agent i solves:
      $U_i^{(k+1)} = \arg\min_{U_i} J_i(X, U_i; \theta_i)$
      s.t. $x_i^{t+1} = f_i(x_i^t, u_i^t)$.
    end for
    Apply $U^{(k+1)}$ to obtain next state $X[t+1]$ using RK4.
    $k \leftarrow k + 1$
  until $d \le d_{\text{term}}$ **or** $k \ge K_{\max}$
end for
return $X = (x^1, \ldots, x^T)$

3.2.3 Forward diffusion process

The forward process typically employs a Markov chain structure with fixed parameters, progressively adding noise to the data until the original data is transformed into pure noise. The conditional distribution at time step t can be expressed as:

$$q\left(X_t \middle| X_{t-1}\right) = N\left(X_t; \sqrt{1-\beta_t} X_{t-1}, \beta_t I\right), \tag{3}$$

where β_t is a predefined noise intensity parameter. Sampling from this distribution, X_t can be represented as:

$$X_t = \sqrt{1-\beta_t} \cdot X_{t-1} + \sqrt{\beta_t} \cdot \epsilon, \tag{4}$$

where $\epsilon \sim N(0, I)$ is random noise from a standard normal distribution. However, this noise-driven process essentially introduces significant randomness and uncertainty, neglecting the continuous sequential nature of the time series, which adversely affects the model's stability and controllability. Therefore, this paper adopts the autoregressive moving average principle to replace the noise injection component in the diffusion model, treating the time series as a continuous evolution process of data points. The goal is to simulate the noise process through a deterministic combination of historical states, thereby mitigating the impact of random disturbances. In Fig. 2, a comparison between existing diffusion-based time series forecasting methods and our method can be observed. Our model makes full use of the inherent characteristics of time series, which exhibit continuous sequential evolution and form a dependency chain characterized by prior information, as represented in the following form:

$$X_t = \sum_{i=1}^{p} \phi_i X_{t-i} + \sum_{j=1}^{q} \theta_j \Delta X_{t-j}, \tag{5}$$

where ϕ_i is the autoregressive coefficient, representing the dependency of the current state on past states. θ_j is the moving average coefficient, indicating the current state's response to historical disturbances. $\Delta X_{t-j} = X_{t-j} - X_{t-j-1}$ represents local changes, simulating the effects of disturbances. The parameters p and q denote the model order, which can be controlled by the length of the sliding window.

By recursive expansion, the conditional distribution X_0 at time step t and the sample $q(X_t|X_0) = N(X_t; \sqrt{\bar{\alpha}_t} X_0, (1-\bar{\alpha}_t)I)$ can be directly obtained from the initial data $X_t = \sqrt{\bar{\alpha}_t} X_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, and t decreases as $\bar{\alpha}_t$ increases. In the forward process of the CARM-Diff model, we rewrite (5) as:

$$X_t = \varphi_t X_0 + \theta_t \cdot \mathrm{Diff}\left(X_t, t\right), \tag{6}$$

where the future sequence X_0 is regarded as the initial state of the forward diffusion process, the historical sequence X_T represents the final state, and the intermediate state X_t indicates the movement of X_0 across t steps, gradually approaching the final state X_T. $\mathrm{Diff}(\cdot)$ denotes the differential operation under the sliding window, used to represent the evolution trend from state X_0 to X_t. By adjusting the coefficients φ_t and θ_t to equal $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1-\bar{\alpha}_t}$, this structure aligns with the diffusion model, thus the deterministic evolutionary trend can be expressed as:

$$\mathrm{Diff}\left(X_t, t\right) = \frac{X_t - \sqrt{\bar{\alpha}_t} X_0}{\sqrt{1-\bar{\alpha}_t}}. \tag{7}$$

From the above content, it can be seen that the forward process of CARM-Diff is a deterministic evolution conditioned on the historical game sequence. We regard the future sequence to be predicted X_0 as the initial state and approach it step by step through a deterministic evolution trend function $\mathrm{Diff}(X_t, t)$ determined by the historical state X_T. The key function $\mathrm{Diff}(X_t, t)$ is not generated randomly, but

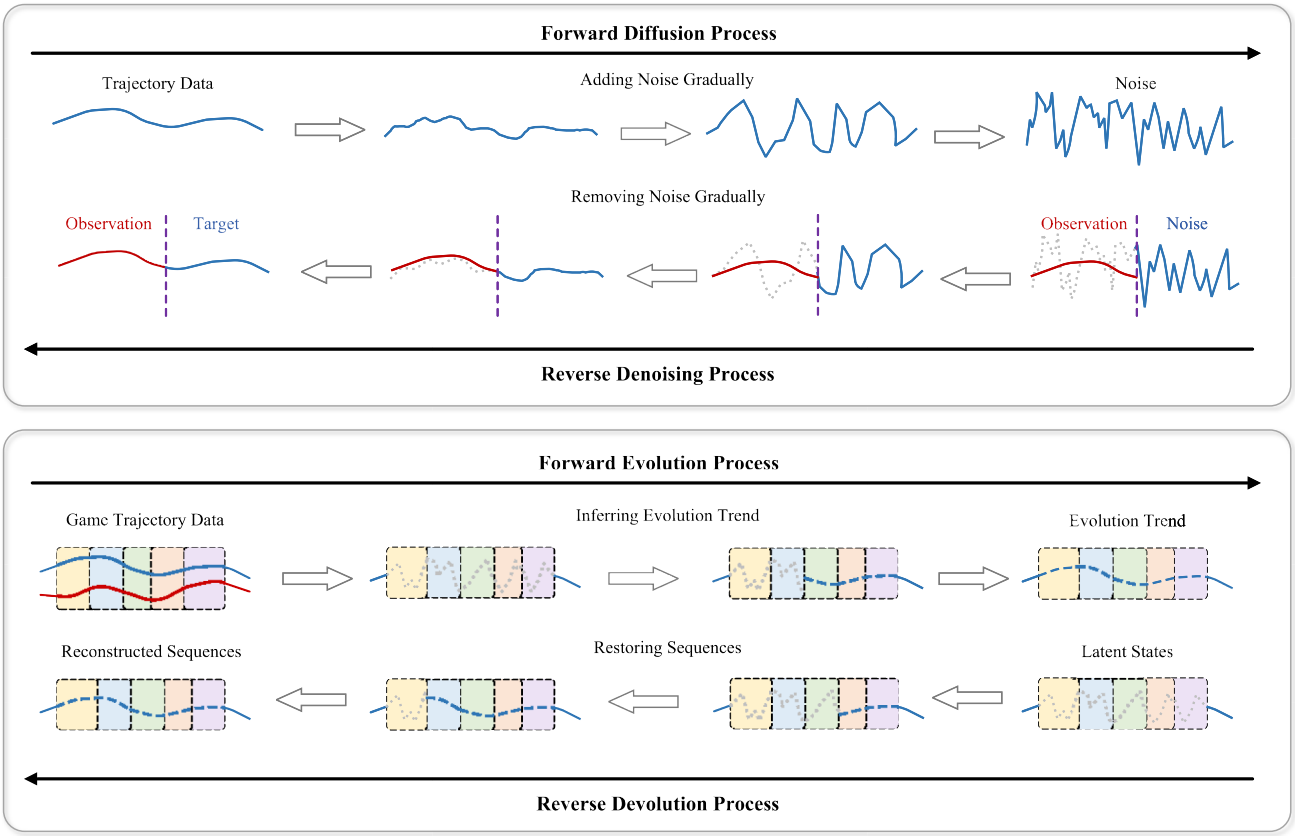

Fig. 2 Comparison between existing diffusion-based methods (above) and CARM-Diff (below)

is entirely computed based on historical game states X_T according to the autoregressive moving average principle. This means that our forward process, conditioned on historical states, simulates a deterministic evolution process constrained by game dynamics from the future state X_0 to the historical state X_T, rather than a random noise process.

Algorithm 2 Training.

Require: Maximum diffusion steps T; Predefined coefficients $\beta_{0:T}$.
1: **repeat**
2: Sample X_0 from the training set;
3: Sample $t \sim \text{Uniform}(1, T)$;
4: Generate the diffusion sample X_t using (6), and calculate the evolution trend Diff (X_t, t) using (7);
5: Use the feature extraction network FE $(\cdot)$ extracts the features $\tilde{X}_t$ and H_t of both players in the game using (10);
6: Use the cross-attention network to fuse the interaction information to obtain $\hat{X}_t$ using (11);
7: Use the gating module to generate the gating parameter γ_t using (14);
8: Use (15) to obtain the predicted sample $\tilde{X}_0\,(X_t, t, \phi)$, and use (16) to obtain the predicted evolution trend $\widetilde{\text{Diff}}\,(X_t, t, \phi)$;
9: Use (17) to calculate the loss L_ϕ;
10: Update the reverse evolution network $\Gamma\,(\phi)$ through gradient descent on $\nabla_\phi L$;
11: **until** converged.

3.2.4 Reverse diffusion process

The reverse process typically employs a Markov chain structure with learnable parameters, gradually recovering the original data from noise. The reverse process at time step t can be modeled as a Gaussian distribution:

$$p_\varphi(X_{t-1} \mid X_t) = N(X_{t-1}; \mu_\varphi(X_t, t), \sigma_t^2 I), \tag{8}$$

where μ_φ is predicted by a neural network, with the goal of matching the posterior mean of the forward process. However, directly predicting the mean μ_φ is quite complex, so it is more common to predict the noise ϵ_φ that is added to the data:

$$\mu_\varphi(X_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(X_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\varphi(X_t, t) \right). \tag{9}$$

Similarly, in the reverse process of CARM-Diff, we substitute the prediction of noise with the prediction of the sequence evolution trend, using the historical series X_T to iteratively generate the future sequence X_0. In each reverse evolution process, the evolution trend Diff (X_0, X_t) is predicted by the

reverse evolution network $\Gamma(\phi)$, reducing the state X_t to X_{t-1}. $\Gamma(\phi)$ solves the predicted value $\tilde{X}_0(X_t, t, \phi)$ of the future sequence X_0 given the intermediate state X_t and the diffusion time step t. In the game scenario, the evolution of the time series not only depends on its own historical states but is also influenced by the actions of adversarial players. Therefore, it is necessary to incorporate the temporal information G_t of the game objects to enhance the model's interactive modeling capabilities. The illustration of the diffusion process in CARM-Diff as shown in Fig. 3.

Let the state of the game object at time step t be represented as $G_t \in \mathbb{R}^{T \times d_g}$ (where d_g is the dimensionality of the features). First, Gaussian noise $\tilde{X}_t = X_t + \epsilon$ is added to the original input X_t. Then, the dynamic features of the game object are obtained through feature encoding:

$$H_t = \mathrm{FE}_g(G_t) \in \mathbb{R}^{T \times d_h}, \tag{10}$$

where FE_g is a learnable feature extraction layer that maps G_t to the same latent space dimension $\tilde{X}_t$ as the current temporal input d_h. The same processing is applied to $\tilde{X}_t$ as well.

The impact of the game object on the original object is extracted through cross-attention, specifically:

$$\hat{X}_t = \mathrm{CrossAttn}(\tilde{X}_t, H_t), \tag{11}$$

the intermediate state $\tilde{X}_t$ of the original object is used as the Query, while the features H_t of the game object serve as the Key-Value. The attention weights are computed, and the interaction information is fused as follows:

$$\mathrm{CrossAttn}(Q, K, V) = \mathrm{Soft\,max}\left(\frac{QK^T}{\sqrt{d_h}}\right)V, \tag{12}$$

where $Q = \mathrm{Linear}_q(\tilde{X}_t)$, $K = \mathrm{Linear}_k(H_t)$, $V = \mathrm{Linear}_v(H_t)$, $Q, K, V \in \mathbb{R}^{T \times d_h}$.

We use a linear module to calculate the difference between the current intermediate state X_t of the original object and the future target sequence X_0. This difference serves to predict the potential distance between the future sequence and the current state. We define the linear transformation $f(\cdot)$, which is updated through parameter optimization in a neural network:

$$f(\tilde{X}_t) = W \cdot \tilde{X}_t + b. \tag{13}$$

The multi-layer perceptron (MLP) is used to dynamically generate gating parameters $\gamma_t \in (0, 1)$, which fuse the updated values with the original input:

$$\gamma_t = \sigma\left(\mathrm{MLP}([\tilde{X}_t \parallel H_t])\right), \tag{14}$$

where $\sigma(\cdot)$ represents the Sigmoid function, which ensures that the gating weights are within the range of [0, 1]. The MLP processes the concatenated input vector through a linear layer to obtain a hidden layer output that has been activated by a nonlinear activation function (such as ReLU). Then, the Sigmoid activation function is applied to map the output of the MLP to the range of [0, 1], resulting in the gating parameter γ_t. This parameter controls the weighted ratio of the updated terms and the original input in the final output.

Finally, by integrating the above equations, we obtain the predicted target state:

$$\tilde{X}_0(X_t, t, \phi) = \gamma_t \cdot f(\tilde{X}_t) + (1 - \gamma_t) \cdot \hat{X}_t, \tag{15}$$

when γ_t approaches 1, it indicates that the updated values from $f(\tilde{X}_t)$ dominate the final output. Conversely, when γ_t approaches 0, it implies that the cross-input $\hat{X}_t$ predominates in the final output.

Once the predicted values of the future sequence $\tilde{X}_0(X_t, t, \phi)$ are obtained, the predicted evolution trend $\widetilde{\mathrm{Diff}}(X_t, t, \phi)$ can be calculated as follows:

$$\widetilde{\mathrm{Diff}}(X_t, t, \phi) = \frac{X_t - \sqrt{\bar{\alpha}_t}\tilde{X}_0(X_t, t, \phi)}{\sqrt{1 - \bar{\alpha}_t}}. \tag{16}$$

The backward process of CARM-Diff aims to gradually recover the future sequence X_0 from the historical state X_T. Each step of this process is conditioned on historical game information and is realized through a conditional backward evolution network $\Gamma(\phi)$. At each step of the backward process, the model receives the current intermediate state X_t while also receiving the historical temporal information G_t of the game opponent. Through a lightweight feature extraction module $FE_g(\cdot)$, we encode G_t into the feature H_t. Then, a cross-attention mechanism is used to explicitly compute the interactive influence between the current state X_t and the opponent's historical features H_t, generating an interaction-enhanced state representation $\hat{X}_t$. Therefore, the prediction of the evolution trend $\widetilde{\mathrm{Diff}}$ and the prediction of the future sequence $\tilde{X}_0$ both heavily depend on the interactive information extracted from the historical states. In short, each step of the backward denoising process is jointly guided by the agent's own historical state and the historical actions of the opponent.

3.3 Training objective

This paper designs a joint loss function based on game equilibrium constraints to simultaneously optimize the evolution trend and strategy Nash equilibrium. Specifically, the total loss function consists of the evolution trend loss term L_{Diff}

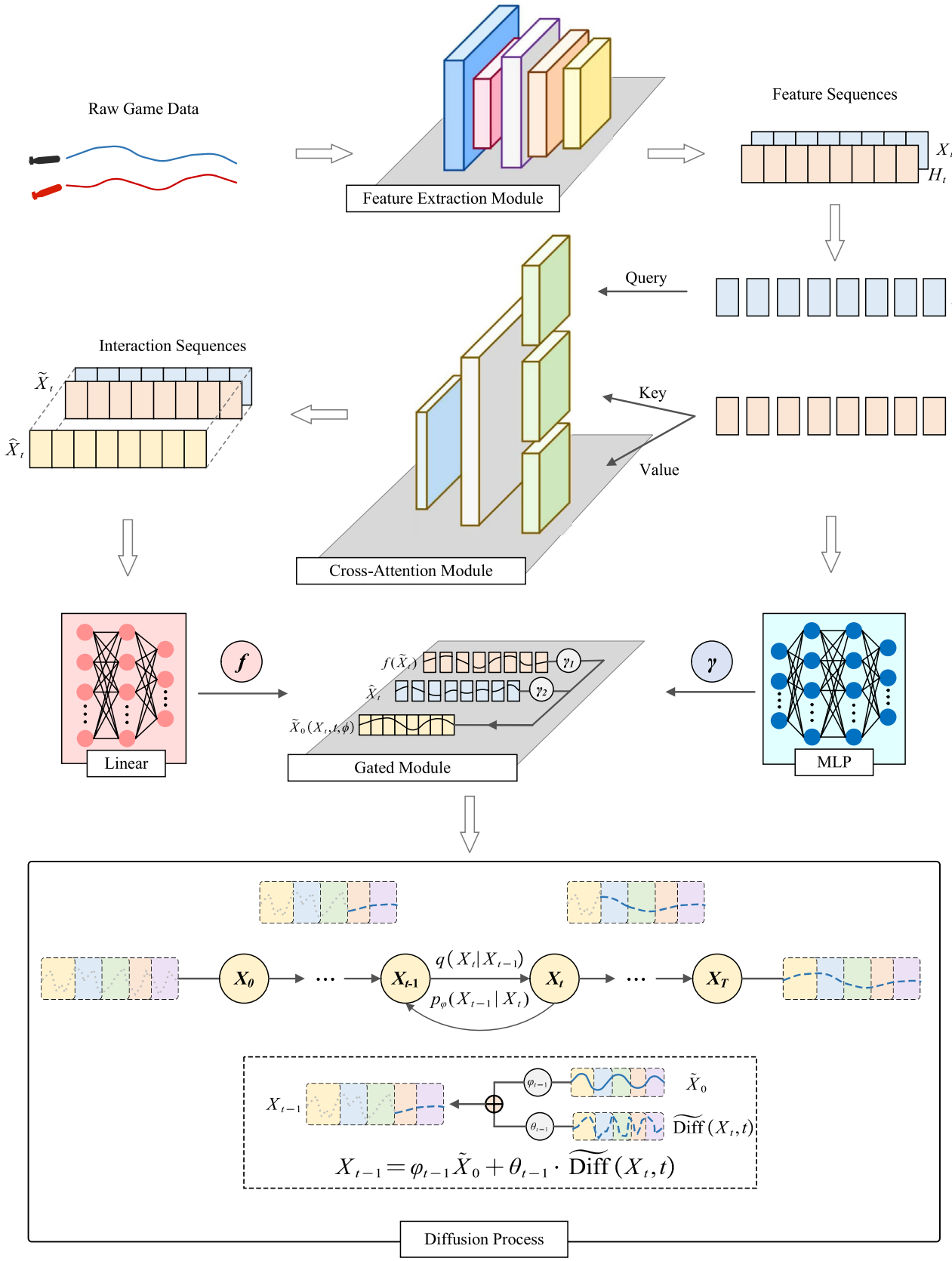

Fig. 3 Data transformation of the diffusion process in CARM-Diff

and the equilibrium penalty term L_{Game}, represented as $L_{\text{total}} = \lambda_1 \cdot L_{\text{Diff}} + \lambda_2 \cdot L_{\text{Game}}$. The specific mathematical expression is given by:

$$L_\phi = E_t \left[\lambda_1 \left\| \text{Diff}\left(X_t, t\right) - \widetilde{\text{Diff}}\left(X_t, t, \phi\right) \right\|_2^2 + \lambda_2 \left\| X_0 - \tilde{X}_0\left(X_t, t, \phi\right) \right\|_2^2 \right], \tag{17}$$

where, the evolution trend loss term $L_{\text{Diff}} = \left\| \text{Diff}\left(X_t, t\right) - \widetilde{\text{Diff}}\left(X_t, t, \phi\right) \right\|_2^2$ is used to maintain supervision over the temporal evolution process, while the equilibrium penalty term $L_{\text{Game}} = \left\| X_0 - \tilde{X}_0\left(X_t, t, \phi\right) \right\|_2^2$ constrains the difference between the model's output future sequence $\tilde{X}_0$ and the actual future sequence X_0. The coefficients λ_1 and λ_2 serve as weighting factors to balance the importance of the two components. For better understanding, the training and forecasting procedures are detailed in Algorithm 2 and 3.

Algorithm 3 Forecasting.

Require: Historical sequence X_T; Trained reverse evolution network $\Gamma(\phi)$; Sampling interval Δt; Predefined coefficients $\beta_{0:T}$.
1: **for** $t = T$ **to** 0 **by** Δt **do**
2: Use the trained interaction network to process the relevant information, referencing training algorithm Steps 4, 5, and 6;
3: Use the reverse evolution network $\Gamma(\phi)$, along with the current state X_t and time t, to obtain the predicted value $\tilde{X}_0\left(X_t, t, \phi\right)$ and the corresponding evolution trend $\widetilde{\text{Diff}}\left(X_t, t, \phi\right)$;
4: Update X_t using (6);
5: **end for**
6: Output the prediction of $\tilde{X}_0$.

4 Experiments

This section includes how we prepare the experimental data, model settings, evaluation metrics, and model performance assessment.

4.1 Data preparation

In this subsection, we establish an underwater vehicle pursuit-evasion model based on iterative optimal responses, clearly defining the objective functions for both the pursuer and the evader. By treating the control variables as strategies, both parties select the optimal strategy that minimizes their respective objective functions in each round of the game, thereby generating a dynamic game trajectory dataset.

4.1.1 Construction of spatial motion model

We consider the motion of an underwater vehicle as the general motion of a rigid body subjected to various external forces in a fluid. A spatial motion model for the underwater vehicle is constructed. By integrating rigid body kinematics, dynamics, and the Newton-Euler equations, along with principles of maneuverability theory, we establish a six degrees of freedom (6-DOF) spatial motion model for the underwater vehicle [28].

Underwater vehicles typically generate control forces only at the rear, resulting in significantly fewer degrees of freedom for control inputs compared to the six degrees of freedom of their spatial motion, categorizing them as typical underactuated systems. The dynamics of underactuated underwater vehicles are described using an inertial frame, where the Earth's sea level is often chosen as the origin of this coordinate system, which is also referred to as the fixed coordinate system. To simplify the description of the AUV's motion model, a non-inertial moving coordinate system that is rigidly attached to the AUV (body-fixed coordinate system) is commonly selected, as shown in Fig. 4. Detailed equations and parameter descriptions can be found in Appendix A.

4.1.2 Data generation

In the pursuit and evasion game, the pursuer and the evader must choose optimal strategies at each time step to maximize their respective objective functions. Based on the forward game trajectory generation method presented in Section 3.1, we design two objective functions to evaluate the performance of each party's strategies. Specifically, the goal of the pursuer is to get as close as possible to the position of the evader while minimizing energy consumption during the movement. The cost function J_p consists of two components: the distance cost to the evader and the energy consumption cost, as shown in (18). The goal of the evader is to distance itself from the pursuer while getting as close as possible to the target point. The cost function J_e for the evader consists of two components: the distance cost to the pursuer and the distance cost to the target point, as shown in (19).

$$J_p = w_p^1 \cdot s_p^1 \cdot \left\| x_p^{t+1} - x_e^t \right\| + w_p^2 \cdot s_p^2 \cdot \left\| x_p^{t+1} - x_p^t \right\|, \tag{18}$$

$$J_e = w_e^1 \cdot s_e^1 \cdot \left\| -\ln\left(x_e^{t+1} - x_p^t + \varepsilon\right) \right\| + w_e^2 \cdot s_e^2 \cdot \left\| x_e^{t+1} - x_d \right\|, \tag{19}$$

where $x_p^t = [x_p^t, y_p^t, z_p^t]^T$ and $x_e^t = [x_e^t, y_e^t, z_e^t]^T$ represent the positions of the pursuer and the evader at time t, respectively, and $x_d = [x_d, y_d, z_d]^T$ denotes the position of the

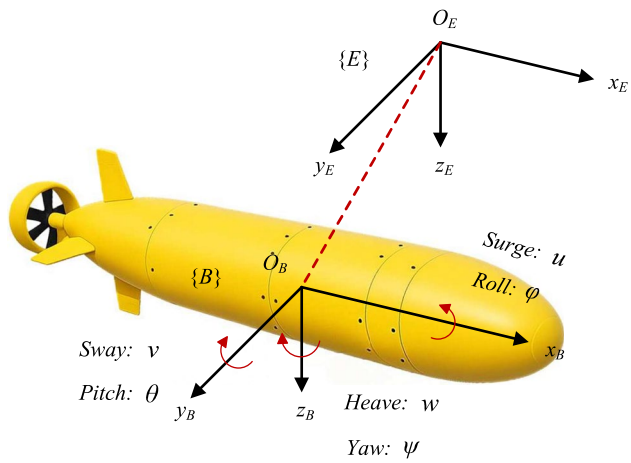

Fig. 4 Fixed Coordinate System {E} and Moving Coordinate System {B}

target point. w_i^j and s_i^j represent the weight coefficients and scaling factors, respectively, while ε is a regularization term (to prevent taking the logarithm of zero), and $\|\cdot\|$ denotes the Euclidean distance. In the experiment, the parameters w_p^1, w_p^2, w_e^1, w_e^2, s_p^1, s_p^2, s_e^1, s_e^2, ε are set to 0.7, 0.3, 0.7, 0.3, 0.5, 0.6, 1.0, 0.1, and 0.1, respectively.

At the start of each game, the initial coordinates of the pursuer x_p^0, the evader x_e^0, and the target position x_d were randomly generated using a uniform distribution within predefined spatial ranges. Specifically, the pursuer's initial coordinates (x_p^0, y_p^0, z_p^0) were sampled from [50, 100] m in each dimension, the evader's initial coordinates (x_e^0, y_e^0, z_e^0) from $[-25, 25]$ m, and the target position (x_d, y_d, z_d) from [30, 60] m. To avoid trivial overlaps, a minimum initial distance of 10 m between the pursuer and the evader was enforced. To ensure consistency between the theoretical motion model and the generated trajectory data, the six-degree-of-freedom equations described in Appendix A are numerically implemented using a fourth-order Runge–Kutta (RK4) integration scheme. In each game iteration, the control inputs $[F_u, \Delta h, \Delta\nu]^T$ are determined by the optimal strategies of the pursuer and evader according to (18)–(19). These inputs are substituted into the dynamic (A2) to update the linear and angular velocities $[u, v, w, q, r]$, as well as the position and attitude variables $[x, y, z, \theta, \psi]$ governed by (A1). The integrated states form a continuous trajectory of each underwater vehicle, which is subsequently discretized into sequential data points. These time-series trajectories are then used as input-output pairs for training and evaluation of the CARM-Diff model described in Section 3. We have established termination conditions for the game, which include: the evader reaching the target point, the pursuer catching the evader, or the game time running out. The game will conclude when any one of these termination conditions is satisfied. The game trajectory generation process is shown as Fig. 5.

The generation of the game trajectories follows the iterative response process outlined in Algorithm 1. The initial control policies for both players are set to zero, and the equilibrium strategy from the previous step is used as the starting state in subsequent time steps. We conducted 2,000 competitive games between the pursuer and the evader, generating a total of 354,679 data points. The duration of each game varies, with the shortest confrontation lasting 42 time steps and the longest lasting 300 time steps, with each time step equivalent to 0.5 seconds. We split the data into training, validation, and test sets in a ratio of 70%, 10%, and 20%, respectively.

4.2 Model setup

The CARM-Diff model is developed based on the PyTorch framework and trained on a single NVIDIA RTX 2060 GPU. We use the Adam optimizer for parameter updates, with a learning rate following a linear growth schedule. Additionally, we employ exponential moving average (EMA) to smooth the parameters and utilize the L1 loss function. The relevant hyperparameters are listed in Table 1.

4.3 Evaluation metrics

To evaluate the performance of CARM-Diff, we use Mean Squared Error (MSE) and Mean Absolute Error (MAE) as assessment metrics. These metrics reflect the deviation between the predicted values and the actual values from different perspectives. The formulas for the evaluation metrics are as follows:

$$\mathrm{MSE} = \frac{1}{\Delta t} \sum_{i=t}^{t+\Delta t} \left(X_0 - \tilde{X}_0\right)^2, \tag{20}$$

$$\mathrm{MAE} = \frac{1}{\Delta t} \sum_{i=t}^{t+\Delta t} \left|X_0 - \tilde{X}_0\right|, \tag{21}$$

the predicted trajectory starts from time t and continues for Δt time steps. The smaller value of the metrics indicates higher accuracy of the model.

4.4 Comparison models

To comprehensively evaluate the performance of the CARM-Diff model in the analysis and prediction of temporal data generated from the game, we select representative comparative models for the comparison experiments. Each of these comparative models has a broad application background and can validate the advantages of our method from different modeling mechanism perspectives.

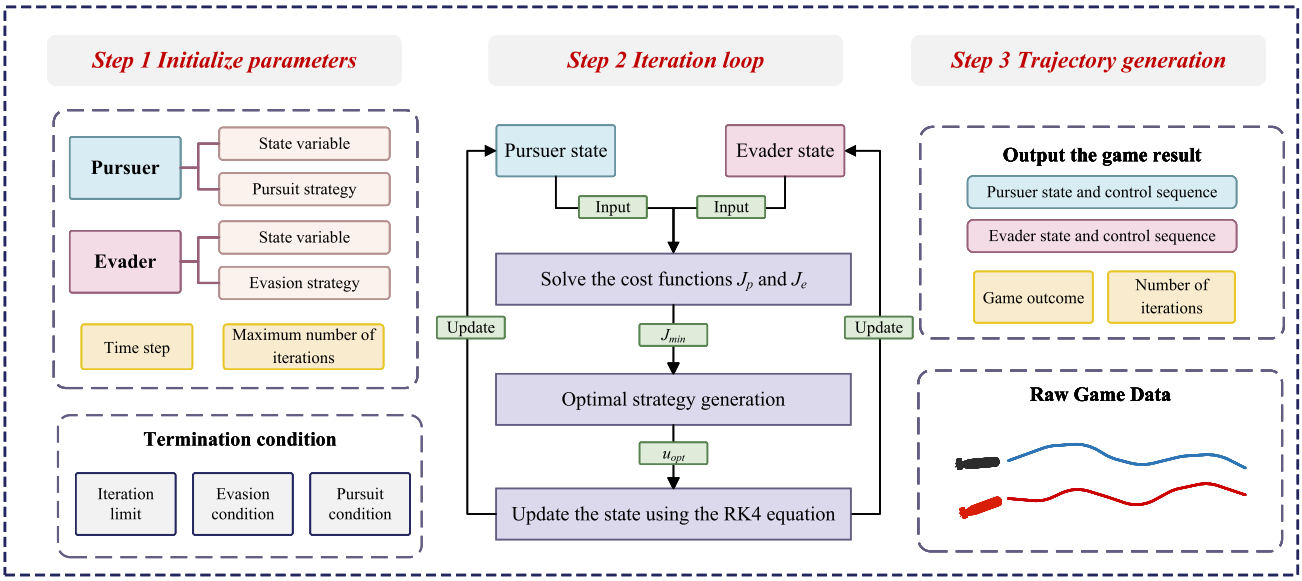

Fig. 5 Game trajectory generation process

Table 1 Experimental hyperparameter settings

Type	Parameter	Value
Parameters in CARM-Diff	Input time steps	(10,20,30,40,50)
	Prediction time steps	(10,20,30,40,50)
	Feature dimension	12
	Reverse sampling steps	1
	Cross-attention heads	4
Adam optimizer	Learning rate	[1e-04,1e-02]
	Momentum parameter	(0.9,0.96)
EMA	Gradient accumulation steps	4
	Decay rate	0.1
	Update interval steps	10
Training configuration	Max epochs	2,000
	Batch size	64

- CARM-Diff-MS (Multi-Step): The model is set up to compare the differences between single-step and multi-step sampling, with a sampling step length of 2 for the 1s time window (not exceeding the window size) and a step length of 5 for other time windows.
- ARMD [4]: ARMD treats the time series as a continuous sequential process evolved from previous data points. It employs chain-based diffusion and priors to accurately model the evolution of the time series, using intermediate state information to enhance the accuracy and stability of the predictions.
- Diffusion-TS [17]: Diffusion-TS utilizes an encoder-decoder transformer with decoupled temporal representations. During training, it directly reconstructs samples and incorporates a Fourier-based loss term to generate high-quality multivariate time series samples.
- iTransformer [6]: iTransformer optimizes the model structure by applying attention mechanisms and feedforward networks to the reversed dimensions. Each time point is embedded as a variable token, effectively capturing the correlations between multivariate data while learning nonlinear representations.
- Dlinear [29]: Dlinear employs a linear model to regress historical time series and directly predict future time series, outperforming most Transformer-based models. The authors consider this a simple yet competitive baseline for studying time series forecasting problems.
- FEDformer [25]: FEDformer combines the Transformer architecture with a seasonal-trend decomposition method to capture the overall profile of the time series, while the Transformer is utilized to capture more detailed structural information.
- Crossformer [26]: Crossformer leverages cross-dimensional dependencies for multivariate time series (MTS) forecasting, utilizing a two-stage attention layer to

efficiently capture these dependencies. It employs information across different levels for accurate predictions.

4.5 Model performance assessment

We conducted comparative experiments between the CARM-Diff model and other models, as well as ablation studies of our own model, along with analyses of the attention mechanism and gating strategies.

4.5.1 Comparative experiment

We achieved time window predictions of 1s, 5s, 10s, and 20s by adjusting the lengths of the input and predicted sequences. In Table 2, we compare the prediction errors of CARM-Diff with other baseline models, with the best results highlighted in bold. Figure 6 illustrates the changes in prediction errors of different models as the time window increases.

The experimental results clearly demonstrate the predictive effectiveness of CARM-Diff across different prediction time spans. CARM-Diff maintains the lowest errors over longer time intervals, particularly in the 10s and 20s time windows, indicating that CARM-Diff possesses a certain degree of accuracy and reliability in long-term time series forecasting. Compared to representative time series diffusion baselines without a cross-attention mechanism, such as ARMD, CARM-Diff has the advantage of explicitly modeling dependencies between agents.

Additionally, increasing the reverse sampling steps leads to a rise in prediction errors and inference time, which we believe is because the reverse process predicts evolution trends rather than random noise. Consequently, the additional information from intermediate states in multi-step reverse sampling is limited, but it accumulates numerical approximation errors in the deterministic evolution chain.

Considering the feasibility of the model in implementing inference and deployment, we replaced the computationally intensive stochastic noise injection with a lightweight feature extraction module and a deterministic diffusion process to control the complexity of the model. This focus on efficiency gives CARM-Diff potential advantages when deployed in resource-constrained edge computing environments, such as onboard computing units in autonomous unmanned systems. In [30], a lightweight random forest model was successfully deployed on edge devices, achieving real-time monitoring of diseases. This demonstrates that a well-designed model can enable efficient inference under strict computational and latency constraints.

We conducted a theoretical analysis of the computational complexity of CARM-Diff: Assuming the input sequence length is T, the feature dimension is d, and the number of reverse sampling steps is S, the dominant computational cost of the model is $O\left(T^2 \cdot d\right)$ from the Cross-Attention module. On the other hand, since the reverse process only employs a single-step backpropagation (S=1), CARM-Diff has a significant advantage in terms of computational load and latency during inference compared to diffusion models that require multi-step sampling.

We compared the training and inference times of various models under the same device and experimental settings as presented in Table 3, with a time window of 5 seconds. The training time refers to the duration of a single complete training session, while the testing time represents the average inference time across the entire test set. It can be observed that CARM-Diff maintains relatively low inference times while also demonstrating superior training

Table 2 Errors comparison of different models

Models	Metrics	1s	5s	10s	20s	Best Count
CARM-Diff	MSE	0.118	0.228	0.307	0.568	7
	MAE	0.209	0.318	0.347	0.548	
CARM-Diff-MS	MSE	0.369	0.423	0.569	1.024	0
	MAE	0.480	0.482	0.518	0.790	
ARMD	MSE	0.140	0.262	0.521	1.123	0
	MAE	0.279	0.272	0.480	0.817	
Diffusion-TS	MSE	0.485	0.985	1.158	1.199	0
	MAE	0.563	0.818	0.900	0.934	
iTransformer	MSE	0.214	0.832	1.077	1.343	0
	MAE	0.223	0.584	0.749	0.900	
DLinear	MSE	0.180	0.451	0.772	0.920	0
	MAE	0.226	0.423	0.650	0.750	
FEDformer	MSE	0.152	0.688	0.836	1.339	0
	MAE	0.178	0.526	0.635	0.903	
Crossformer	MSE	0.133	0.544	0.749	0.905	1
	MAE	0.176	0.455	0.608	0.726	

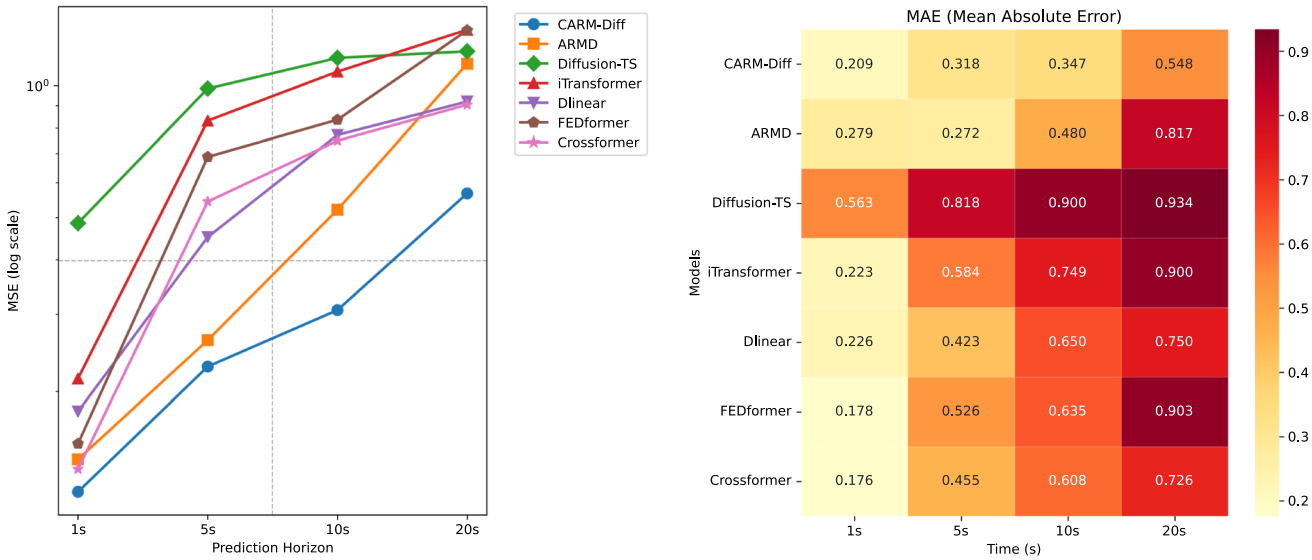

Fig. 6 Errors of different models

Table 3 Training/inference time (s) of different models

Models	Training Time	Inference Time
CARM-Diff	48.41	13.077
ARMD	13.32	15.376
Crossformer	56.83	15.960
DLinear	23.11	18.890
Diffusion-TS	38.27	86.730
iTransformer	66.85	113.905
FEDformer	134.20	110.755

efficiency compared to most complex Transformer models. We believe that the efficiency advantage of CARM-Diff lies in the fact that single-step backpropagation avoids the linear overhead of multiple sampling, lightweight feature extraction replaces large-scale spatiotemporal stacking, and cross-attention is computed only once within a single step.

4.5.2 Ablation experiment

To evaluate the contributions of various components within the proposed CARM-Diff, we conducted ablation experiments. We constructed several variants of CARM-Diff by systematically removing or modifying key components to assess their impact on prediction performance over different time spans. The logical framework for constructing the variants is as follows:

- CARM-Diff (GNN-Interaction): The variant replaces the cross-attention module with a Graph Interaction Network (GIN) module. This module constructs a fully connected graph between the pursuer and evader, where node features are updated via multi-head message passing.
- CARM-Diff (Additive-Fusion): This variant replaces the residual gated fusion mechanism with a direct additive fusion strategy. Specifically, the output feature is computed as the element-wise sum of the self-evolution term and the cross-interaction term without adaptive weighting, i.e., $\tilde{X}_0 = f(\tilde{X}_t) + \hat{X}_t$.
- CARM-Diff (Concat-Fusion): This variant fuses the two terms through concatenation followed by a linear projection, i.e., $\tilde{X}_0 = W[f(\tilde{X}_t) \parallel \hat{X}_t] + b$, thereby assessing the influence of simple feature-level fusion.
- CARM-Diff w/o DetEvolution (without Deterministic Evolution): The variant replaces the deterministic evolution trend with a standard Gaussian diffusion mechanism. Specifically, instead of constructing the intermediate states through a deterministic sliding transition, we adopt a stochastic diffusion formulation:

$$X_t = \sqrt{\bar{\alpha}_t} X_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon, \quad \varepsilon \sim N(0, I). \tag{22}$$

The model remains identical in architecture, loss design, and optimization settings; only the forward transition from X_0 to X_t is altered. The L_{Diff} is achieved by the error

between the actual noise ε and the estimated noise $\hat{\varepsilon}$ inferred from $\tilde{X}_0$. This variant serves to isolate the contribution of the deterministic evolution component by removing its temporal inductive bias and evaluating the model under purely stochastic noise conditions.

- CARM-Diff w/o FeatEnc(without Feature Encoding Module): This variant removes the feature extraction module that represents the input sequence.
- CARM-Diff w/o CrossAttn(without Cross-Attention Module): This variant replaces the cross-attention mechanism module, which models the dependencies between game objects, with a linear layer.
- CARM-Diff w/o Gate(without Gate Module): This variant removes the gating module that fuses the feature information from both parties in the game.

The results of the ablation experiments are presented in Table 4. These results highlight the importance of each component in CARM-Diff. To further validate the advantage of the proposed cross-attention mechanism in capturing inter-agent interactions, we replaced it with a Graph Interaction Network (GIN) that models information propagation between the pursuer and the evader using message passing on a fully connected two-node graph. Although the GIN-based variant also achieves reasonable prediction accuracy, its performance is inferior to that of the cross-attention version. This demonstrates that attention-guided interaction modeling provides better temporal alignment and adaptivity to dynamic strategy changes compared with graph-based relational modeling.

Regarding the variant that lacks a deterministic evolution trend, it is evident that the standard Gaussian diffusion variant performs worse than the original model across all prediction windows. We believe this is because the deterministic forward process explicitly injects temporal evolution priors into the time series, reducing variance and suppressing error diffusion. In contrast, while random diffusion is more general, it is more prone to accumulating noise under long-range dependencies and strategic coupling in adversarial games.

Removing the feature encoding module results in the model's inability to effectively extract crucial information from the input sequence, leading to a significant decline in performance over short time spans, indicating its critical role in short-term predictions. The removal of the cross-attention module hinders the model's ability to effectively capture the dependencies between game objects, with performance degradation being particularly pronounced in longer time frame predictions. Additionally, the three variants of the gating mechanism (w/o Gate, Additive-Fusion, and Concat-Fusion) exhibit higher prediction errors across all time windows. Therefore, removing or replacing the gating module prevents the model from effectively weighting and adjusting the feature contributions of different game participants, leading to less flexible information coupling and a noticeable increase in prediction errors. The proposed residual gated structure, by contrast, adaptively learns a time-varying fusion factor γ_t through an MLP-based gating generator, enabling dynamic balance between self-evolutionary and interaction-induced features. These results confirm that the gating mechanism plays a crucial role in maintaining the stability and accuracy of trajectory prediction. Overall, feature encoding, cross-attention, gating module, and deterministic evolution trends are all key components of the CARM-Diff model. The absence or substitution of any of these modules affects the model's performance to varying degrees, demonstrating the synergistic effect of these components in enhancing prediction accuracy.

Table 4 Errors comparison of ablation experiment

Models	Metrics	1s	5s	10s	20s	Best Count
CARM-Diff	MSE	0.118	0.228	0.307	0.568	8
	MAE	0.209	0.318	0.347	0.548	
CARM-Diff (GNN-Interaction)	MSE	0.218	0.379	0.505	0.692	0
	MAE	0.358	0.464	0.590	0.633	
CARM-Diff (Additive-Fusion)	MSE	0.291	0.334	0.586	0.845	0
	MAE	0.417	0.429	0.542	0.697	
CARM-Diff (Concat-Fusion)	MSE	0.318	0.345	0.564	0.805	0
	MAE	0.440	0.437	0.534	0.672	
CARM-Diff w/o DetEvolution	MSE	0.624	0.655	0.761	1.056	0
	MAE	0.627	0.633	0.685	0.829	
CARM-Diff w/o FeatEnc	MSE	0.283	0.430	0.515	0.756	0
	MAE	0.401	0.499	0.535	0.665	
CARM-Diff w/o CrossAttn	MSE	0.127	0.290	0.481	0.906	0
	MAE	0.221	0.362	0.460	0.733	
CARM-Diff w/o Gate	MSE	0.368	0.430	0.506	0.722	0
	MAE	0.473	0.506	0.524	0.635	

4.5.3 Attention mechanism and gating strategies

We conducted a detailed analysis of the attention mechanism and gating strategies used in the CARM-Diff model, focusing on the entropy variations of the attention heads and the dynamic changes of the gating factor γ. Through this analysis, we gained a better understanding of how the model gradually learns the importance of different features during the training process and how it effectively adjusts the fusion methods between various features.

Variations in attention entropy During the training process of the model, the entropy values of the attention heads exhibit significant dynamic changes, reflecting the concentration level of the attention distribution: higher entropy indicates a more dispersed attention distribution, while lower entropy suggests a stronger focus on specific positions. By monitoring the entropy changes of the four attention heads, we can observe the results shown in Fig. 7. The Head 0 has a relatively high entropy value in the early stages of training, indicating a more dispersed attention distribution that gradually stabilizes. This suggests that the attention mechanism of this head balances its contributions across multiple input positions during the learning process. In contrast, Head 3 exhibits a low and stable entropy value, maintaining consistent focus on specific positions throughout training, demonstrating stronger concentration. Meanwhile, Heads 1 and 2 display volatility in their entropy values, indicating that they continually adjust their attention patterns to adapt to different input features. Overall, as training progresses, the model's attention mechanism becomes more concentrated on key information, leading to a gradual decrease in entropy, which reflects improved information processing capabilities.

Dynamic variation of gating factor γ The dynamic variation of the gating factor γ in the model controls the degree of fusion between the original features and the enhanced features obtained through the cross-attention mechanism. Through the gating mechanism, the model adaptively adjusts the fusion process based on the characteristics of the input data, as illustrated in Fig. 8. Initially, the value of γ is relatively low, indicating that the model relies more on the feature information provided by the cross-input. As training progresses, γ gradually increases, reflecting the model's introduction of information from the updating process. The eventual stabilization of the γ value suggests that the model has achieved a stable balance between the contributions of cross-input information and the updating process. This stability indicates that the model has learned to flexibly adjust the contribution ratios of these two types of information according to their varying importance. Through this analysis, we can see the significance of the gating mechanism in the model, as the variation of the gating factor illustrates the model's gradual learning of how to fuse information in specific ways to optimize the final output.

5 Conclusion

This paper proposes a model named CARM-Diff for solving inverse trajectory games by inferring unknown parameters in the game from historical data generated by non-cooperative games. A lightweight feature extraction module is

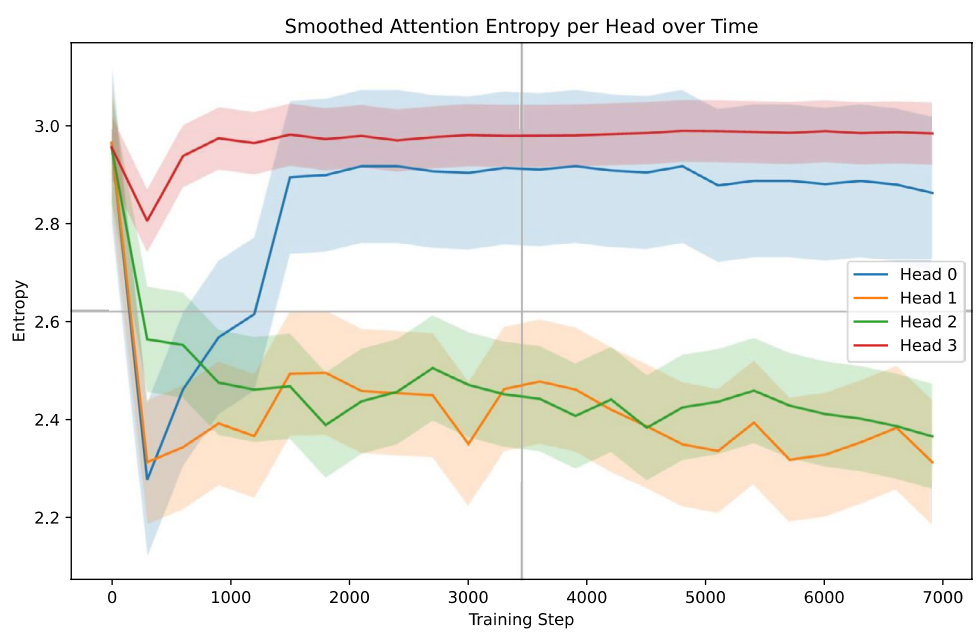

Fig. 7 Variation in attention entropy

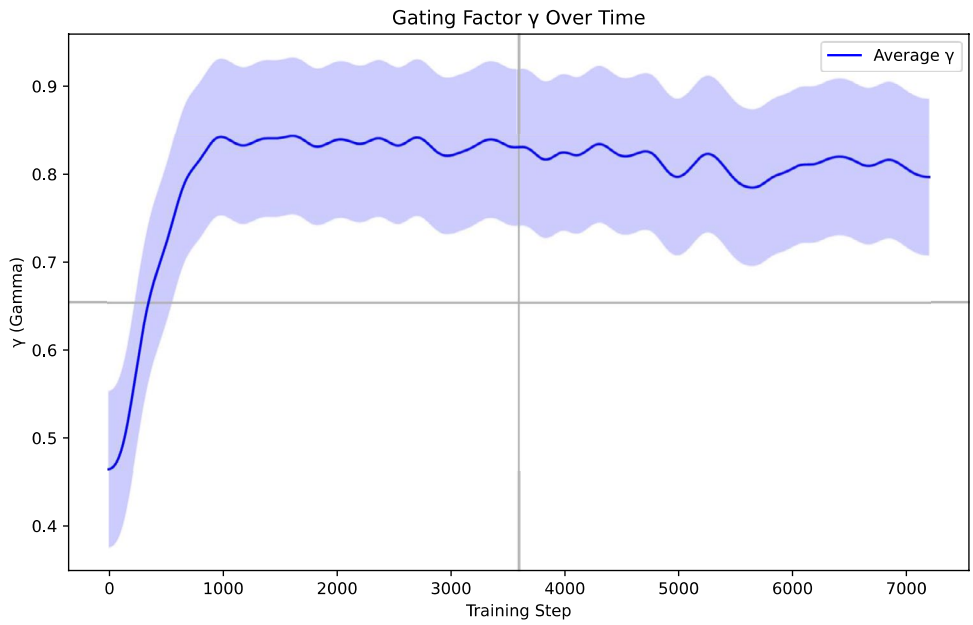

Fig. 8 Variation in gating factors

constructed to capture local temporal features, and a deterministic combination of historical states is used to simulate the noise process, thereby reducing the impact of stochastic disturbances. Considering the interactive information brought by the game entities, a cross-attention mechanism is introduced into the diffusion process to model the temporal interaction relationships between game participants and their opponents. Additionally, we propose a residual gated trajectory modeling structure that integrates the linear evolution trends of the game participants with the feature information of their opponents to achieve adaptive information fusion. Through the design of simulation experiments to generate the game dataset, CARM-Diff demonstrates superior accuracy in recovering game parameters and predictive capability compared to other advanced methods.

Moreover, the methodological framework of CARM-Diff is not limited to underwater environments. Its capability to model interactive strategy evolution and temporal dynamics makes it applicable to various real-world domains involving multi-agent coordination and competition. For instance, it can enhance trajectory forecasting in autonomous driving, improve strategy modeling in aerial or ground robotics, and support predictive decision-making in security and defense simulations. Therefore, the broader significance of this study lies in contributing a general-purpose diffusion-based paradigm for dynamic multi-agent systems.

Future research could consider applying CARM-Diff to more complex multi-agent game environments, such as dynamic alliances and incomplete information games, to enhance the model's adaptability to real-world complex gaming scenarios. The current cross-attention mechanism and gating structure are specifically designed for bilateral interactions, which may not fully capture the higher-order dependencies that emerge in games involving three or more agents. In such cases, the number of pairwise attention computations increases quadratically with the number of agents, leading to higher computational costs and potential training instability. Future work will focus on extending CARM-Diff to scalable multi-agent diffusion frameworks, such as hierarchical or graph-based attention mechanisms, to efficiently model many-to-many strategic interactions while maintaining computational tractability. Additionally, although CARM-Diff has demonstrated good parameter recovery capabilities in the inverse trajectory game problem, it currently focuses mainly on inferring a subset of unknown parameters in the game and has not yet escalated

Table 5 Main parameter symbols of AUV

DOF	States	Position/Attitude	Spatial Velocity	Force/Torque
1	Surge	x	u	$X(T_x)$
2	Sway	y	v	$Y(T_y)$
3	Heave	z	w	$Z(T_z)$
4	Roll	φ	p	$K(M_x)$
5	Pitch	θ	q	$M(M_y)$
6	Yaw	ψ	r	$N(M_z)$

to modeling and understanding the strategies of game participants. Future studies could further explore the transition from parameter inference to strategy reasoning, attempting to construct a framework that can implicitly or explicitly model the strategy evolution processes of game entities. We will aim to improve our work based on the aforementioned prospects.

Appendix A: Motion model and parameters

In the appendix, we provide the detailed kinematic and dynamic equations of the AUV, along with descriptions of the relevant parameters.

Based on rigid body kinematics and the Newton-Euler equations, the kinematic model of the underwater vehicle is established as follows:

$$\begin{cases} \dot{\varphi} = p + q\sin\varphi\tan\theta + r\cos\varphi\tan\theta \\ \dot{\theta} = q\cos\varphi - r\sin\varphi \\ \dot{\psi} = q\sin\varphi/\cos\theta + r\cos\varphi/\cos\theta \\ \dot{x} = u\cos\psi\cos\theta + v(\cos\psi\sin\theta\sin\varphi - \sin\psi\cos\varphi) \\ \quad + w(\cos\psi\sin\theta\cos\varphi + \sin\psi\sin\varphi) \\ \dot{y} = u\sin\psi\cos\theta + v(\sin\psi\sin\theta\sin\varphi + \cos\psi\cos\varphi) \\ \quad + w(\sin\psi\sin\theta\cos\varphi - \cos\psi\sin\varphi) \\ \dot{z} = -u\sin\theta + v\cos\theta\sin\varphi + w\cos\theta\cos\varphi. \end{cases} \tag{A1}$$

Based on the fundamental force relationships acting on the underwater vehicle in motion, the dynamics model of the AUV is established as follows:

$$\begin{cases} m_{11}\dot{u} = m_{22}vr - m_{33}wq - X_u u - X_{u|u|}u|u| + \tau_u \\ m_{22}\dot{v} = m_{33}wp - m_{11}ur - Y_v v - Y_{v|v|}v|v| + \tau_v \\ m_{33}\dot{w} = m_{33}uq - m_{22}vp - Z_w w - Z_{w|w|}w|w| + \tau_w \\ m_{44}\dot{p} = (m_{22} - m_{33})\,vw + (m_{55} - m_{66})\,qr - K_p p \\ \quad - K_{p|p|}p|p| - \overline{BG}W\cos\theta\sin\varphi + \tau_p \\ m_{55}\dot{q} = (m_{33} - m_{11})\,uw + (m_{66} - m_{44})\,pr - M_q q \\ \quad - M_{q|q|}q|q| - \overline{BG}W\sin\theta + \tau_q \\ m_{66}\dot{r} = (m_{11} - m_{22})\,uv + (m_{44} - m_{55})\,pq - N_r r \\ \quad - N_{r|r|}r|r| + \tau_r. \end{cases} \tag{A2}$$

since the AUV is configured not to roll, the roll angle p and roll rate p are both set to 0. Here, $[u, v, w, q, r, x, y, z, \theta, \psi]^T$ represents the state variables of the underwater vehicle, and $[F_u, \Delta h, \Delta v]^T$ represents the control variables. The state variables are 10-dimensional, including linear velocity, angular velocity, position, and attitude angles. The control variables are 3-dimensional and directly affect the acceleration terms in the dynamic equations, including thrust and torque control variables, as shown in Table 5. The kinematic and dynamic (A1)–(A2) were numerically integrated using the RK4 method to ensure numerical stability and accuracy in simulating the underwater vehicle's motion. Control forces and torques were computed from the optimal pursuit–evasion strategies at each time step and applied to update the state vector $[u, v, w, q, r, x, y, z, \theta, \psi]^T$. The resulting temporal evolution of positions and orientations directly formed the trajectory sequences used in Section 4.1 for dataset construction and model validation.

Author Contributions Conceptualization: ZZ, LL. Methodology: ZZ, LL. Investigation: ZZ, WT and YC. Funding acquisition: LL. Project administration: LL, ZZ. Supervision: CJ. Writing - original draft: ZZ. Writing - review & editing: ZZ, LL.

Data Availability The data presented in this study are available upon request from the corresponding author.

Declarations

Competing Interests The authors have no competing interests to declare that are relevant to the content of this article.

Ethics Approval and Consent to Participate The authors have no competing interests to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Yang Y, Xiao Y, Li T (2021) A survey of autonomous underwater vehicle formation: performance, formation control, and communication capability. IEEE Commun Surv Tutor 23(2):815–841
2. Zhang B, Ji D, Liu S, Zhu X, Xu W (2023) Autonomous underwater vehicle navigation: a review. Ocean Eng 273:113861
3. Chen Y, Shi Y, Dai X, Meng Q, Yu T (2025) Pursuit-evasion game with online planning using deep reinforcement learning. Appl Intell 55(6):1–17
4. Gao J, Cao Q, Chen Y (2025) Auto-regressive moving diffusion models for time series forecasting. Proc AAAI Conf Artif Intell 39:16727–16735
5. Xu W, Li R, Du X, Li B, Xing L (2025) Trajectory prediction via proposal guided transformer with out way attention. Sci Rep 15(1):13513
6. Liu Y, Hu T, Zhang H, Wu H, Wang S, Ma L, Long M (2024) itransformer: inverted transformers are effective for time series forecasting. In: The twelfth international conference on learning representations

7. Zhang J, Zhu Z, Xue Y, Deng Z, Qin H (2024) Local path planning of under-actuated auv based on vadwa considering dynamic model. Ocean Eng 310(Part2):12
8. Liu X, Peters L, Alonso-Mora J (2023) Learning to play trajectory games against opponents with unknown objectives. IEEE Robot Autom Lett 8(7):4139–4146
9. Le Cleac'h S, Schwager M, Manchester Z (2021) Lucidgames: online unscented inverse dynamic games for adaptive trajectory prediction and planning. IEEE Robot Autom Lett 6(3):5485–5492
10. Li W, Zhang Y, Li L, Lv Y, Wang M (2024) A pedestrian trajectory prediction model for right-turn unsignalized intersections based on game theory. IEEE Trans Intell Transp Syst 25(8):9643–9658
11. Geng M, Cai Z, Zhu Y, Chen X, Lee D-H (2023) Multimodal vehicular trajectory prediction with inverse reinforcement learning and risk aversion at urban unsignalized intersections. IEEE Trans Intell Transp Syst 24(11):12227–12240
12. Yang B, Lu Y, Wan R, Hu H, Yang C, Ni R (2024) Meta-irlsot++: a meta-inverse reinforcement learning method for fast adaptation of trajectory prediction networks. Expert Syst Appl 240(000):12
13. Wu H, Zhai D-H, Xia Y (2025) Probabilized dynamic movement primitives and model predictive planning for enhanced trajectory imitation learning. IEEE Trans Industr Electron 72(1):620–628
14. Li H, Jiao H, Yang Z (2023) Ais data-driven ship trajectory prediction modelling and analysis based on machine learning and deep learning methods. Trans Res E Logist Transport Rev 175:103152
15. Luo J, Xiao Y, Li Y, Xiao Y, Yao W (2025) Multimodal deep learning framework for vessel trajectory prediction. Ocean Eng 336:121766
16. Ho J, Jain A, Abbeel P (2020) Denoising diffusion probabilistic models. Adv Neural Inf Process Syst 33:6840–6851
17. Yuan X, Qiao Y (2024) Diffusion-TS: Interpretable diffusion for general time series generation. In: The twelfth international conference on learning representations.
18. Wei L, Hu P, Feng R, Feng H, Du Y, Zhang T, Wang R, Wang Y, Ma Z-M, Wu T (2024) Diffphycon: a generative approach to control complex physical systems. In: The thirty-eighth annual conference on neural information processing systems
19. Wei L, Feng H, Yang Y, Feng R, Hu P, Zheng X, Zhang T, Fan D, Wu T (2025) CL-diffphycon: closed-loop diffusion control of complex physical systems. In: The thirteenth international conference on learning representations
20. Li H, Xing W, Jiao H, Yang Z, Li Y (2024) Deep bi-directional information-empowered ship trajectory prediction for maritime autonomous surface ships. Transport Res Part E Logist Transport Rev 181:103367
21. Yu W, Kok SY, Srivastava G (2025) Emoiplanner: human emotion and intention aware socially acceptable robot navigation in human-centric environments. Expert Syst 42(2):13718
22. Chen C, Chen X, Yang Y, Hang P (2024) Sparse attention graph convolution network for vehicle trajectory prediction. IEEE Trans Vehic Technol
23. Zhang D, Chen C, Tan F, Qian B, Li W, He X, Lei S (2023) Multi-view and multi-scale behavior recognition algorithm based on attention mechanism. Front Neurorobot 17:1276208
24. Pu Z, Wang H, Liu Z, Yi J, Wu S (2022) Attention enhanced reinforcement learning for multi agent cooperation. IEEE Trans Neural Netw Learn Syst 34(11):8235–8249
25. Zhou T, Ma Z, Wen Q, Wang X, Sun L, Jin R (2022) Fedformer: frequency enhanced decomposed transformer for long-term series forecasting. In: International conference on machine learning. PMLR, pp 27268–27286
26. Zhang Y, Yan J (2023) Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting. In: The eleventh international conference on learning representations
27. Wood DA (2024) Spatio-temporal attributes of varicella-zoster case number trends assist with optimizing machine learning predictions. Medinformatics 1(2):43–53
28. McCue L (2016) Handbook of marine craft hydrodynamics and motion control [bookshelf]. IEEE Control Syst Mag 36(1):78–79
29. Zeng A, Chen M, Zhang L, Xu Q (2023) Are transformers effective for time series forecasting? Proc AAAI Conf Artif Intell 37:11121–11128
30. Murugesan S, Naganathan A (2024) Edge computing-based cardiac monitoring and detection system in the internet of medical things. Medinformatics

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.